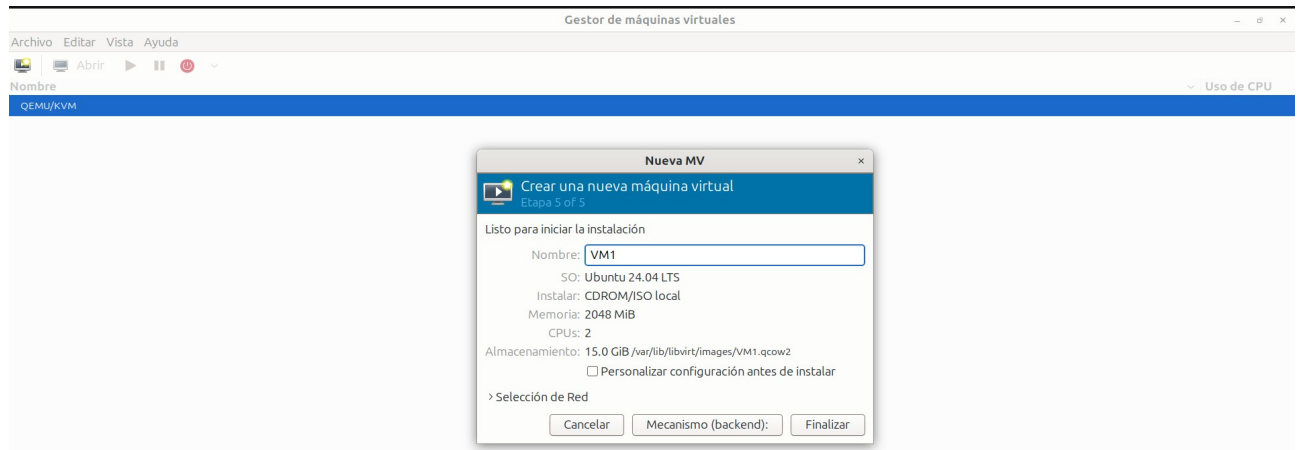


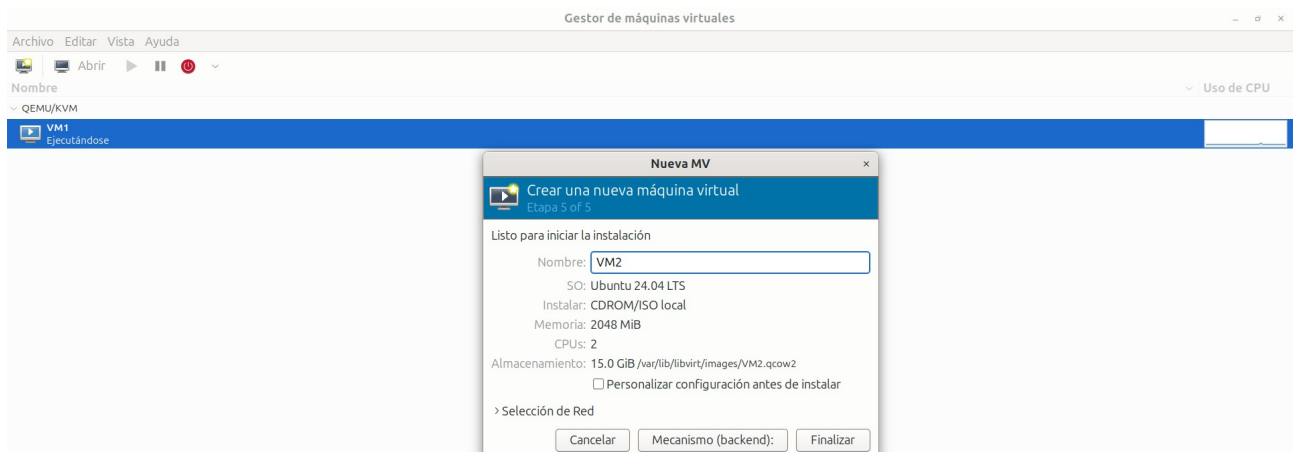
CAPTURAS DE PRUEBA Y EVIDENCIA FUNCIONAL

Maquinas en KVM

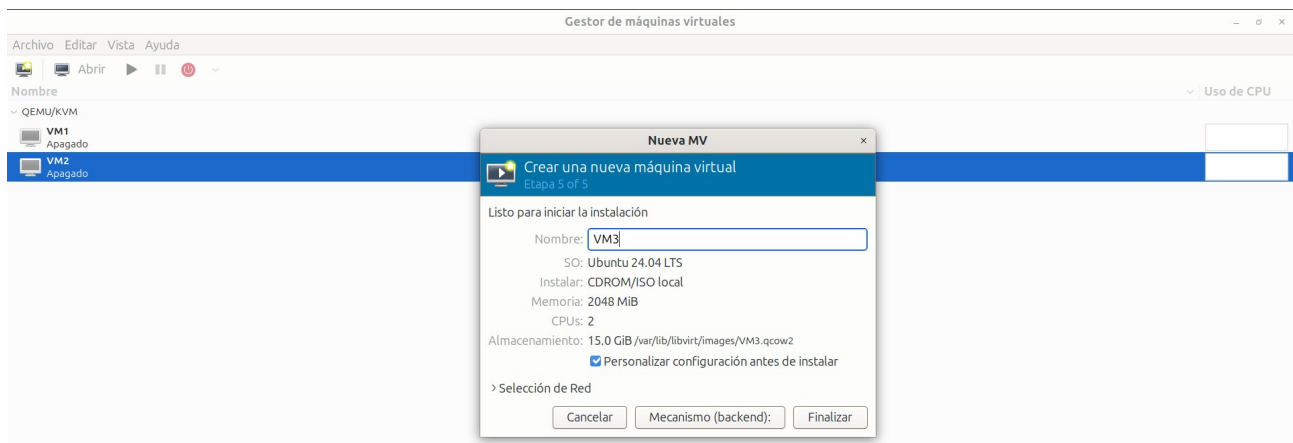
VM1



VM2



VM3



Containerd, buildkit en vm1

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
pablo@vm1:~/tmp$ hostname
containerd --version
nerdctl --version
sudo nerdctl run --rm alpine:latest ping -c 2 google.com
vm1
containerd github.com/containerd/containerd/v2 2.2.1
nerdctl version 2.3.5
PING google.com (142.251.215.14): 56 data bytes
64 bytes from 142.251.215.14: seq=0 ttl=116 time=79.451 ms
64 bytes from 142.251.215.14: seq=1 ttl=116 time=93.993 ms

--- google.com ping statistics ---
2 packets transmitted, 2 packets received, 0% packet loss
round-trip min/avg/max = 79.451/86.722/93.993 ms
pablo@vm1:~/tmp$
```

Podman en vm2

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
pablo@vm2:~$ hostname
podman --version
podman run --rm docker.io/library/alpine:latest echo "Podman funciona correctamente en VM2"
podman run --rm docker.io/library/alpine:latest ping -c 2 google.com
vm2
podman version 4.9.3
Podman funciona correctamente en VM2
PING google.com (142.251.215.110): 56 data bytes

--- google.com ping statistics ---
2 packets transmitted, 0 packets received, 100% packet loss
pablo@vm2:~$
```

Docker en vm3

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
pablo@vm3:~$ sudo docker --version
systemctl is-active docker
systemctl is-enabled docker
sudo docker run hello-world
Docker version 29.7.2, build a7dcaa6
active
enabled

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Comunicación API1 con API2

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  2  bash - api2 + - [ ] ...

pablo@vm1:~/proyecto-sopes/api2$ curl http://localhost:8082/api2/202112092/call-api1
curl http://localhost:8081/api1/202112092/call-api2
{"apiname":"API1","message":"The API1 located on the VM1 is working","connection":true,"carnet":"202112092"}
{"apiname":"API2","message":"The API2 located on the VM1 is working","connection":true,"carnet":"202112092"}
pablo@vm1:~/proyecto-sopes/api2$
```

Comunicación de API3

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  1

pablo@vm2:~$ echo "==== API3 -> API1 ====="
curl -s http://localhost:8083/api3/202112092/call-api1
echo
echo

echo "==== API3 -> API2 ====="
curl -s http://localhost:8083/api3/202112092/call-api2
echo
==== API3 -> API1 =====
{"apiname":"API1","message":"The API1 located on the VM1 is working","connection":true,"carnet":"202112092"}

==== API3 -> API2 =====
{"apiname":"API2","message":"The API2 located on the VM1 is working","connection":true,"carnet":"202112092"}
```

Zot-catalogo en mv3

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  1

pablo@vm3:~$ echo "=== CATALOGO ZOT ==="
curl -s http://localhost:5000/v2/_catalog
echo

echo "=== API1 ==="
curl -s http://localhost:5000/v2/api1-202112092/tags/list
echo

echo "=== API2 ==="
curl -s http://localhost:5000/v2/api2-202112092/tags/list
echo

echo "=== API3 ==="
curl -s http://localhost:5000/v2/api3-202112092/tags/list
echo
=== CATALOGO ZOT ===
{"repositories":["api1-202112092","api2-202112092","api3-202112092"]}
=== API1 ===
{"name":"api1-202112092","tags":["v1"]}
=== API2 ===
{"name":"api2-202112092","tags":["v1"]}
=== API3 ===
{"name":"api3-202112092","tags":["v1"]}
pablo@vm3:~$
```

Comunicaciones de API1 y API2, pruebas hechas desde vm1

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  2

pablo@vm1:~$ echo "==== API1 -> API2 ====="
curl -s http://localhost:8081/api1/202112092/call-api2
echo
echo
...
echo
echo

echo "==== API2 -> API1 ====="
curl -s http://localhost:8082/api2/202112092/call-api1
echo
echo

echo "==== API2 -> API3 ====="
curl -s http://localhost:8082/api2/202112092/call-api3
echo
==== API1 -> API2 =====
{"apiname":"API2","message":"The API2 located on the VM1 is working","connection":true,"carnet":"202112092"}

==== API1 -> API3 =====
{"apiname":"API3","message":"The API3 located on the VM2 is working","connection":true,"carnet":"202112092"}

==== API2 -> API1 =====
{"apiname":"API1","message":"The API1 located on the VM1 is working","connection":true,"carnet":"202112092"}

==== API2 -> API3 =====
{"apiname":"API3","message":"The API3 located on the VM2 is working","connection":true,"carnet":"202112092"}
```

Comunicaciones con API3, pruebas hechas desde vm2

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  1

● pablo@vm2:~$ echo "==== API3 -> API1 ====="
curl -s http://localhost:8083/api3/202112092/call-api1
echo
echo

echo "==== API3 -> API2 ====="
curl -s http://localhost:8083/api3/202112092/call-api2
echo
==== API3 -> API1 =====
{"apiname":"API1","message":"The API1 located on the VM1 is working","connection":true,"carnet":"202112092"}

==== API3 -> API2 =====
{"apiname":"API2","message":"The API2 located on the VM1 is working","connection":true,"carnet":"202112092"}
```

Reservas de DHCP

```
pablo@LMALEDUCADA2: ~  
pablo@LMALEDUCADA2:~$ virsh -c qemu:///system domifaddr VM1  
virsh -c qemu:///system domifaddr VM2  
virsh -c qemu:///system domifaddr VM3  
Name          MAC address      Protocol    Address  
-----  
vnet1         52:54:00:b0:c8:76  ipv4        192.168.122.101/24  
Name          MAC address      Protocol    Address  
-----  
vnet2         52:54:00:bf:45:e0  ipv4        192.168.122.22/24  
Name          MAC address      Protocol    Address  
-----  
vnet0         52:54:00:f7:1c:e2  ipv4        192.168.122.239/24  
pablo@LMALEDUCADA2:~$ virsh -c qemu:///system domiflist VM1  
virsh -c qemu:///system domiflist VM2  
virsh -c qemu:///system domiflist VM3  
Interface     Type      Source      Model    MAC  
-----  
vnet1         network   default     virtio    52:54:00:b0:c8:76  
Interface     Type      Source      Model    MAC  
-----  
vnet2         network   default     virtio    52:54:00:bf:45:e0  
Interface     Type      Source      Model    MAC  
-----  
vnet0         network   default     virtio    52:54:00:f7:1c:e2
```